Stanford University
Electrical Engineering

Introduction to VLSI Systems (EE271)
SIMD Accelerator - Assignment #1

Contents

1 Introduction **3**

2 Setting Up Your Environment **3**

2.1 Setting Up Caddy 3

2.2 Setup 4

3 The Basic Structure of Our SIMD Accelerator **5**

3.1 The Processing Element 5

3.2 The Memory Banks 6

3.3 The Controller 7

3.4 Why Use a SIMD Accelerator? 8

4 Assignment #1 **9**

4.1 Files 9

4.2 Your Task 9

4.3 Testing Your Implementation 10

4.4 Submission 10

1 Introduction

For the EE271 class project, you will be working on a Single Instruction Multiple Data (SIMD) accelerator for matrix multiplication. This project is segmented into three components. In the first component you will complete a Python simulator of the accelerator. In the second component of the project you will complete an RTL implementation of the accelerator using the lessons learned from completing the Python simulator and synthesize your RTL implementation in order to determine its power performance and area (PPA). In the third component of the project you will optimize your RTL implementation in order to improve its PPA.

This document will focus on the details/instructions needed to complete the first component of the project, which we have labeled **Assignment #1**. Please read the entirety of this document prior to starting the assignment. Section 2 will cover setting up your development environment. Section 3 will cover the overall architecture of the SIMD accelerator. This information is vital to properly implementing the instruction set of the accelerator within the Python simulator. Finally, Section 4 will cover what needs to be done to finish the assignment as well as how to submit the assignment once your implementation is complete.

2 Setting Up Your Environment

2.1 Setting Up Caddy

- If you have never logged into caddy before, the first step would be to initiate an ssh session with the command:

```
$ ssh <yoursunetid>@login.farmshare.stanford.edu
```

- Next, Create a Farmshare desktop instance on OnDemand.
- If this is your first time using OnDemand for this quarter or if you have terminated your previous sessions:
  - Click on "Interactive Apps" tab in the menu bar on the top
  - Click on FarmShare Desktop in the drop down
  - Click on Launch in the next window. If it is your first time, it will take sometime to launch
- If it is not your first time using OnDemand and if you have not killed the desktops you launched earlier:
  - Click on the "My Interactive Sessions" icon next to "Interactive Apps" tab in the top bar

  - Here you should see your old session. Click on Launch FarmShare Desktop button

- Once the desktop launches, right click on your mouse and click on "Open in Terminal"
- From this Farmshare terminal, ssh to caddy cluster using:

```
$ ssh -XY caddy.best.stanford.edu
```

- If you face any issues, please check for any relevant information on FAQs page. If you cannot resolve them, please post on Ed or come to Office Hours

2.2 Setup

Clone the project with:

```
$ git clone git@code.stanford.edu:EE271/simd-part1.git
$ cd simd-part1
```

Make sure to use tcsh when working the project directory:

```
$ kinit
$ aklog
$ tcsh
$ source load_modules.sh
```

If you don't have `conda` installed, download the binary with the command:

```
$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Then follow the steps in this link. Once you have the directory cloned and `conda` installed, you have to set up your `conda` environment. This can be done with the following command:

```
$ conda env create -p ee271-conda-env -f requirements.yml -vv
```

You can then activate this conda environment using:

```
$ conda activate ./ee271-conda-env
```

Your environment should now be configured. Make sure to activate your `conda` environment every time you open a new terminal.

3 The Basic Structure of Our SIMD Accelerator

3.1 The Processing Element

The basic building block of any SIMD accelerator is the processing element. The processing element is the component where the actual computation occurs. As seen in Fig. 1 the processing element takes as input two 32-bit integer data operands and an instruction. The instruction controls what operation the PE performs on the input data. Additionally the PE has an internal state element called an accumulation register.

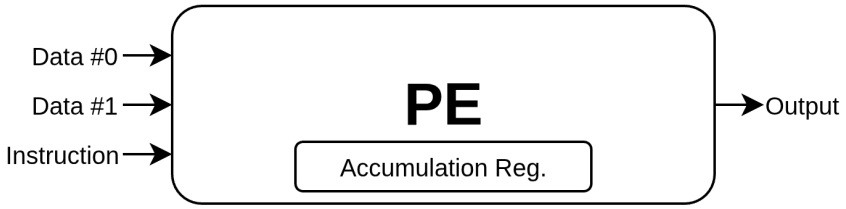


Figure 1: A diagram showcasing the basic inputs and outputs of a processing element.

The instruction input to the PE has three fields: `Opcode`, `Mode`, and `Value`. Let us begin by discussing the function of the `Mode` field of the instruction. One trend in modern accelerator design is the use of smaller bit-widths for data representation. Specifically the use of 16-bit or even 8-bit representations alongside the more traditional 32-bit data representations. To accommodate differing bit-widths with our accelerator we will allow the input 32-bit data operands to be split into either two 16-bit operands or four 8-bit operands. The purpose of the `Mode` field is to inform the PE whether it should treat the inputs as 32-bit operands, two 16-bit operands, or four 8-bit operands. When the PE is using a mode other than 32-bit, the accumulation register is segmented into the requisite number of *virtual accumulation registers* to allow for each pair of *virtual inputs* to have its own accumulation register.

All lanes represent signed two's-complement integers. The supplied tests use 32-bit input and output words and a 64-bit accumulator: four 16-bit accumulator lanes in INT8, two 32-bit lanes in INT16, or one 64-bit lane in INT32. The least-significant input lane corresponds to the least-significant accumulator and output lanes; the remaining lanes follow in the same order. Perform arithmetic independently in each lane.

Next, we will discuss the purpose of the `Opcode` field of the instruction. The Opcode, encodes one of the 6 supported instructions: `MAC`, `NOP`, `OUT`, `PASS`, `CLR`, and `RND`. These instructions perform the following functions as dictated below:

`MAC`: This opcode indicates a multiply-and-accumulate operation should be performed. The PE multiplies the two data operands and adds the result into the accumulation register. If the `Mode` field of the instruction indicates the PE is in 8-bit mode, then four

simultaneous multiply-and-accumulate operations are performed on each of the four 8-bit input operands. Use the full signed product when adding to the corresponding wide accumulator lane. MAC does not saturate.

`NOP:` This opcode indicates no operation should be performed by the PE.

`OUT:` Clamp each signed accumulator lane to its signed output range, then narrow and concatenate the lanes in their existing order. For output lane width $w \in \{8, 16, 32\}$ and signed accumulator value A, the output value is

$$O' = \min(\max(A, -2^{w-1}),\ 2^{w-1} - 1).$$

OUT preserves the accumulation register. Clamping affects only the value written to the output.

`PASS:` This opcode indicates that the contents of the `Data0` input should be written directly to the accumulation register. If the PE is operating in a mode other than 32-bit then each of the *virtual inputs* associated with `Data0` are written to the *virtual accumulation registers*. Here `Data0` is input A, the broadcast vector from MEM1; input B is the per-PE matrix word from MEM0. Sign-extend each Data0 lane separately into its corresponding accumulator lane.

`CLR:` This opcode sets the value of the accumulation register and the output register to zero.

`RND:` Divide each signed accumulator lane by 2^s and round to the nearest integer, with ties to even. Interpret the five-bit `Value` field as the unsigned $s \in \{0, \ldots, 31\}$. Write the rounded value back to the same wide accumulator lane and preserve the previous output register.

Among PE instructions, only OUT and CLR update the output register. MAC, PASS, RND, and NOP preserve its previous value.

3.2 The Memory Banks

Now that we have a processing element capable of performing calculations, we now must consider how we can feed this processing element with data. This task is accomplished through a set of memory banks: `MEM0`, `MEM1`, and `MEM2`. The exact structure of these memory banks relative to the processing elements can be seen in Fig. 2.

Inside the accelerator these three memories are grouped into a single logical component called the *Main Buffer*. This component takes an instruction as input which controls what actions it performs. This instruction has four fields: `Opcode`, `Mode`, `MemAOffset`, and `MemBOffset`. Similarly to the `Mode` field in the PE instruction the `Mode` field in the memory instruction informs the Main Buffer what mode it is operating in. The `Opcode` field encodes one of three possible instructions, described below:

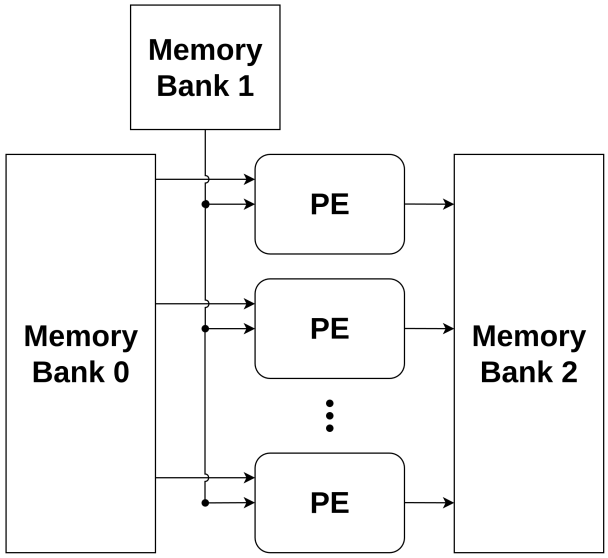


Figure 2: A diagram showcasing the basic memory structure of the accelerator.

`READ`: When operating in 32-bit mode this opcode indicates that `MEM0` should read out the value at the address pointed to by `MemAOffset` and `MEM1` should read out the value at the address pointed to by `MemBOffset`. When operating in 16-bit mode the functionality of `MEM0` remains the same, however, `MEM1` should read out the value at `MemBOffset[MSB:1]` and `MemBOffset[0]` should be used to select between the upper 16-bits or the lower 16-bits of the data. Whichever piece of data is selected is then repeated twice to fill out the whole 32-bit word. When operating in 8-bit mode the functionality of `MEM0` remains the same, however, `MEM1` should read out the value at `MemBOffset[MSB:2]` and `MemBOffset[1:0]` should be used to select between the four 8-bit pieces of the data. This piece of data is then repeated four times to fill out the whole 32-bit word.

`WRITE`: This instruction indicates that the output data from the PEs should be written to `MEM2` at the address pointed to by `MemAOffset`.

`NOP`: This instruction indicates no action should be taken.

Note: If you do not understand why we have structured the `READ` instruction in this way, please refer to the supplementary lecture where we discuss mapping matrix-vector multiplication onto this accelerator.

3.3 The Controller

Now that we have an array of processing elements and a buffer to feed them data, we will now focus on how to control all of the moving parts. This task is accomplished by the *Controller*.

The exact structure of the controller relative to the other components of the accelerator can be seen in Fig. 3.

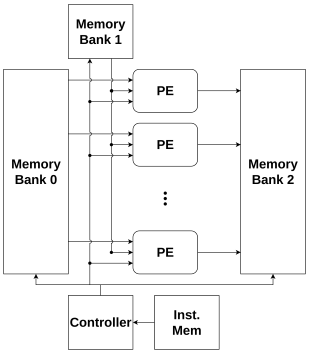


Figure 3: A diagram showcasing the basic structure of the SIMD accelerator.

The controller takes an instruction as input from an instruction memory. This instruction has five fields: `MemInstruction`, `PEInstruction`, `Count`, `MemAIncrement`, and `MemBIncrement`. The fields `MemInstruction`, and `PEInstruction` have the same structure as the main buffer instructions and the processing element instructions described above. The controller then passes the `MemInstruction`, and the `PEInstruction` to the main buffer, and processing elements respectively. It is important to note that since this is a **SIMD** accelerator each of the processing elements receive the same instruction (single instruction) but each of those processing elements operates on different data from memory bank 0 (multiple data). This process of passing instructions to the processing elements and main buffer is then repeated `Count + 1` times. However, for the memory instruction the `MemAOffset` and `MemBOffset` fields are incremented by `MemAIncrement` and `MemBIncrement` each time the instruction is repeated.

3.4 Why Use a SIMD Accelerator?

Given the above description of the accelerator you might begin to wonder why we would use such a device. The number one reason is the ease of expandability. You can add as many or as few processing elements as you desire without any major architectural changes. This allows SIMD arrays to be easily scaled to tackle larger and large problems as they arise. The second reason is ease of routability. Given that each processing element receives the exact same instruction, we don't need to perform any complicated routing to get an instruction to its processor. This approach helps to drastically reduce the routing congestion within a design.

4 Assignment #1

4.1 Files

src This directory contains all of the code which will implement the accelerator simulator.

- **accelerator.py** This is the file where you will implement the top-level of your accelerator.
- **assembler.py** This file contains the implementation of the assembler which converts assembly into machine code for the accelerator. You **do not** need to edit this file.
- **compiler.py** This file contains the code which converts a matrix multiplication into accelerator assembly. You **do not** need to edit this file.
- **instruction.py** This file contains the class definitions which organize the various instruction formats present in the accelerator into classes for ease of use. You **do not** need to edit this file, however, you may find it useful to refer to this file when implementing each component of the accelerator.
- **main_buffer.py** This is the file where you will implement the main buffer.
- **processing_element.py** This is the file where you will implement the processing element.

test This directory contains the public sanity tests for your implementation.

- **Makefile** This is a Makefile which will allow you to easily run all of the tests.
- **test_accelerator.py** This is a file which will run full tests on the accelerator. Passing this test is a good indication that your implementation of the simulator is fully functional.
- **test_main_buffer.py** This is a file which will allow you to run tests on `main_buffer.py` by itself. Passing this test is a good indication that your implementation of the main buffer is functioning correctly.
- **test_processing_element.py** This file contains basic PE tests in INT8, INT16, and INT32. You are welcome to expand this test to cover other operating regimes as you see fit.

4.2 Your Task

Your task will be to complete the Python simulator of the SIMD accelerator by completing the implementation of the following files: `processing_element.py`, `main_buffer.py`, and `accelerator.py`. The portions of the code which need to be edited can be found by looking through the aforementioned files for instances of the phrase: "`Your Code Here`". This design is meant to be expandable, as such many parameters of the design (such as the bit-widths of various components) are determined through a configuration object passed to the constructor.

4.3 Testing Your Implementation

To test your implementation, you can enter the `test` directory and run the command: `make run_tests`. This will run all provided tests on your implementation. There are also makefile rules to run each test individually. If you wish to use these rules please consult the makefile. We **highly recommend** that you unit test your design

4.4 Submission

To receive full credit, you must meet the following criteria.

1. You must pass all tests. You can check if you meet this requirement by running `make run_tests` in the `test` director.
2. You must also pass private staff tests using additional input values and instruction sequences that follow the specification above. Passing the provided tests alone does not establish correctness.

Once you believe you meet both of the above requirements you can generate a zip file for submission by running the following at the top-level of the git repository:

```
$ make generate_submission
```

Submit to Gradescope by **Friday, Nov 13, 11:59 PM PST**.